



**unab**

UNIVERSIDAD NACIONAL  
GUILLERMO BROWN

# CLASE 4 - Unidad 2

## Funciones

ALGORITMOS Y ESTRUCTURAS DE DATOS (184)

## AGENDA

- **Temario:**
  - Conceptos de función. Parámetros de entrada y salida.
  - Tipos de pasaje de parámetros. Variables locales y globales.
  - Tiempo de vida. Anidamiento de procedimientos y funciones.
  - Visibilidad y alcance. Implementación en unidades.
- **Ejemplos en Lenguajes Python**
- **Temas relacionados y links de interés**
- **Práctica**
- **Cierre de la clase**

**Función:** Es una estructura que permite abstraer funcionalidad de un programa. Está compuesta por sentencias o instrucciones que permiten resolver una determinada tarea.

Una función permite:

- ❑ **Definir** nuestras propias funciones y usarlas.
- ❑ Definir un bloque de **código reutilizable** que puede ejecutarse muchas veces dentro del programa.
- ❑ Crear soluciones **modulares** sin repetir código.
- ❑ **Reducir** el número de líneas de código en el programa.

**Función:** tiene dos partes.

- 1.- **Definición** de la función
- 2.- **Invocación** de la función.

## Sintaxis de la Definición de una Función sin parámetros

1. Palabra clave **def**
2. Nombre de la función.
3. Paréntesis **()**.
4. Termina en dos puntos **“:”**
5. Bloque de **código**

ENCABEZADO

CUERPO

```
1 def sumar():  
2     print(8+4)
```

ENCABEZADO

CUERPO

```

1  def sumar():
2      print(8+4)

```

← ENCABEZADO

← CUERPO

Con este código *definimos* la función **sumar** pero no esta completo, la idea es usar la función creada.

```

1  def sumar():
2      print(8+4)
3
4  sumar()

```

← INVOCACIÓN

Con este código *definimos* la función **sumar** y además *invocamos* la función definida, lo que nos permite que sea ejecutada por python.

Paso a paso de la ejecución:

---

```

→ 1 def sumar():
  2     print(8+4)
  3
  4 sumar()
    
```

---

La ejecución del programa comienza en la primer línea  
Se cargan las funciones definidas y el flujo de ejecución  
pasa automáticamente a la primer línea ejecutable del  
programa

---

```

→ 1 def sumar():
  2     print(8+4)
  3
→ 4 sumar()
    
```

---

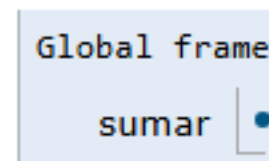
[Edit this code](#)

En la línea 4 se encuentra con la  
**Invocación** de la función **sumar()**,  
así que el flujo de ejecución salta a la línea donde se encuentra la definición  
de la función **sumar()**

Print output (drag lower right corner to resize)

Frames

Objects



function  
sumar()

```
→ 1 def sumar():
    2     print(8+4)
    3
→ 4 sumar()
```

[Edit this code](#)

El flujo de ejecución se encuentra en la función, comienza a ejecutarse el cuerpo de la función

```
→ 1 def sumar():
→ 2     print(8+4)
    3
    4 sumar()
```

[Edit this code](#)

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

sumar

function  
sumar()

sumar

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

sumar

function  
sumar()

sumar

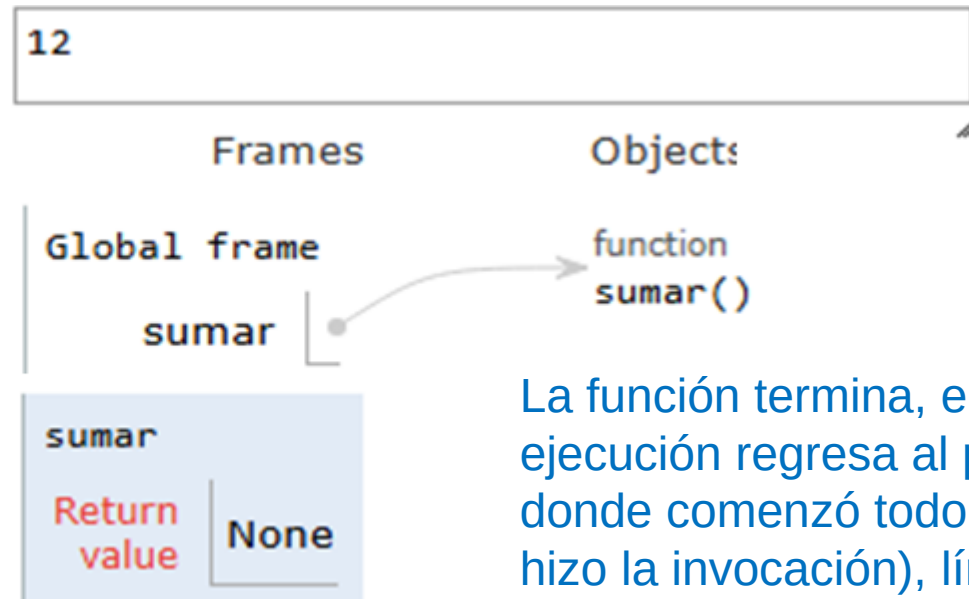


```

1 def sumar():
2     print(8+4)
3
4     sumar()
    
```

[Edit this code](#)

→ line that just executed  
→ next line to execute



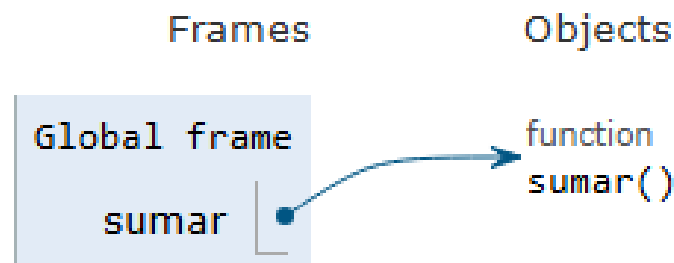
Print output (drag lower right corner to resize)

12

```

1 def sumar():
2     print(8+4)
3
4     sumar()
    
```

[Edit this code](#)



El flujo de ejecución regresa al programa Principal y termina.

## Sintaxis de la Definición de una Función con parámetros

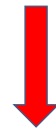
1. Palabra clave **def**
2. Nombre de la función.
3. Paréntesis(**parámetros**).
4. Termina en dos puntos “:”
5. Bloque de **código**

ENCABEZADO

CUERPO

```
9  def sumar(a,b):#parametros formales
10     print(a+b)#sumamos los parametros
```

Los **parámetros** los usamos como en Matemáticas, separados por comas y es importante el orden



PARÁMETROS

```
9  def sumar(a,b):#parametros formales ← ENCABEZADO
10      print(a+b)#sumamos los parametros ← CUERPO
11      #invocación
12      sumar(8,4)#parametros reales ← INVOCACIÓN
```

En este código los parámetros toman dos **valores** reales(8 y 4), los cuales se machean con el **encabezado** teniendo en cuenta el orden que ocupan. El valor 8 pasa a ser el primer parámetro y el valor 4 pasa a ser el segundo parámetro dentro de la función

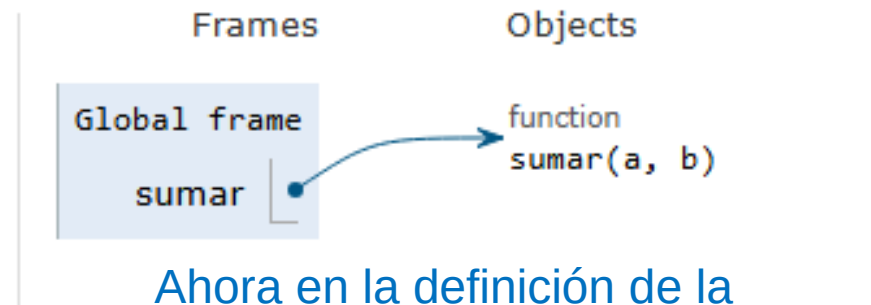
```
sumar(a,b): #parametros formales
sumar(8,4)  #parametros reales
```

```
1 #2.-funciones con parametros
2 #definición
→ 3 def sumar(a,b):#parametros formales
4     print(a+b)#sumamos los parametros
5 #invocación
6 sumar(8,4)#parametros reales
```

El flujo de ejecución comienza al principio del programa, encuentra la función la guarda y pasa a la primer línea de ejecución del programa.

```
1 #2.-funciones con parametros
2 #definición
→ 3 def sumar(a,b):#parametros formales
4     print(a+b)#sumamos los parametros
5 #invocación
→ 6 sumar(8,4)#parametros reales
```

Print output (drag lower right corner to resize)



Ahora en la definición de la función aparecen los parámetros

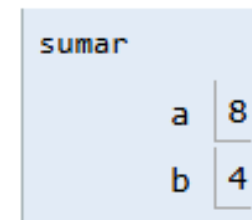
# Clase 4: CONCEPTOS DE FUNCIÓN

```

1 #2.-funciones con parametros
2 #definición
→ 3 def sumar(a,b):#parametros formales
4     print(a+b)#sumamos los parametros
5 #invocación
→ 6 sumar(8,4)#parametros reales
    
```

→ line that just executed  
→ next line to execute

Print output (drag lower right corner to resize)



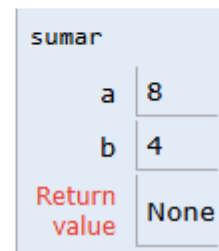
La función sumar se activa  
Y se guardan los valores de los parámetros

```

1 #2.-funciones con parametros
2 #definición
3 def sumar(a,b):#parametros formales
→ 4     print(a+b)#sumamos los parametro:
5 #invocación
6 sumar(8,4)#parametros reales
    
```

Print output (drag lower right corner to resize)

12



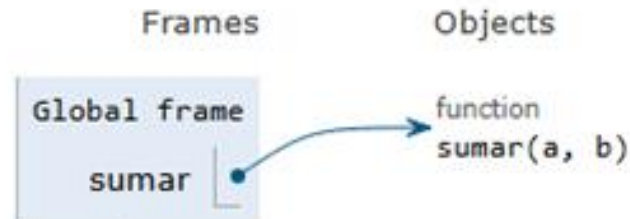
La función termina su ejecución, imprime el resultado y devuelve el control al programa o punto donde fue invocada.

Python 3.11  
[known limitations](#)

```
1 #2.-funciones con parametros
2 #definición
3 def sumar(a,b):#parametros formales
4     print(a+b)#sumamos los parametros
5 #invocación
6 → sumar(8,4)#parametros reales
```

Print output (drag lower right corner to resize)

12



El flujo de ejecución  
vuelve al programa y  
este termina, ya no hay  
código para ejecutarse.

Que pasa si necesitamos datos que son calculados dentro de la función, como hacemos?? Si queremos usar el valor calculado en la función para hacer otra cuenta

## Uso del return en Funciones

- ❖ La palabra clave **return** se usa dentro del cuerpo de una función.
- ❖ Se usa siempre como **ultima línea** del cuerpo de la función ya que cuando se ejecuta nos saca automáticamente de la misma, regresando el flujo de ejecución al punto de invocación de la función.
- ❖ Permite **sacar valores** del cuerpo de la función.

```
#uso de return
def pi(): #defino una funcion
    #retorna siempre el mismo valor
    x = 3.14159 #creo una variable con el valor
    return x #retornamos el valor de la variable
#invoco
pi()
```

## EJEMPLO DE FUNCIONES

```
1
2  # Area de funciones
3
4  def saludar(nombre):
5      print("Hola, " + nombre + "!")
6
7  def sumar(a, b):
8      return a + b
9
10
11  # Programa principal
12
13  nombre = input("¿Cuál es tu nombre? ")
14  saludar(nombre)
15
16  num1 = float(input("Introduce el primer número: "))
17  num2 = float(input("Introduce el segundo número: "))
18
19  print( sumar(num1, num2))
20  print( multiplicar(num1, num2))
21
```



## EJEMPLO DE FUNCIONES

```
#funciones

def area_triangulo(a,b):
    area=((a*b)/2)
    return (area)

#programa principal

altura=int(input('Ingresar altura del triangulo: '))
base=int(input('Ingresar base del triangulo: '))
x= area_triangulo(altura,base)
print ( "El área del triángulo es: ", x)

print("calcula otro area")
al=int(input('Ingresar altura del triangulo: '))
ba=int(input('Ingresar base del triangulo: '))
print ()
print ( "El área del triángulo es: ", area_triangulo(al,ba))
```

## Clase 4: CONCEPTOS DE FUNCIÓN

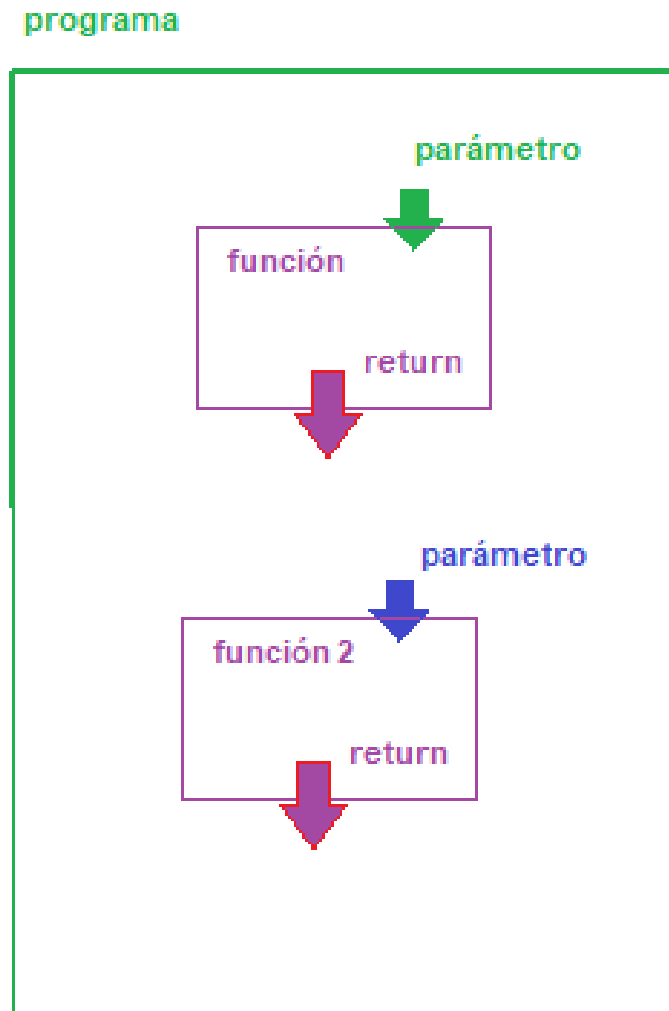
### EJEMPLO DE FUNCIONES

```
1
2  # Area de funciones
3
4  def sumar(a, b):
5      |   return a + b
6
7  def sumari(a, b):
8      |   print (a + b)
9
10 # Programa principal
11
12
13 num1 =int (input("Introduce el primer número: "))
14 |
15 num2 = int (input("Introduce el segundo número: "))
16
17 print( sumar(num1, num2))
18 print()
19 print("la suma de los dos númeos es: ", sumar(num1, num2))
20 print ()
21 sumari(num1, num2)
22
```

The diagram illustrates the execution flow of the code. A red arrow points from the `sumar` function definition (lines 4-5) to its first call in `print( sumar(num1, num2))` (line 17). An orange arrow points from the `sumari` function definition (lines 7-8) to its call in `sumari(num1, num2)` (line 21). A light blue arrow points from the `sumar` function call (line 17) to the `print` statement (line 19), indicating that the result of the function is passed to `print`. Another light blue arrow points from the `sumari` function call (line 21) to the end of the program (line 22), indicating that the function is executed but does not return a value.

La forma de intercambiar  
datos entre las funciones es  
a través del programa.

El programa le provee datos  
a la función usando los  
**parámetros** y las funciones  
retornan resultados  
obtenidos al programa con el  
**return**



### En resumen:

- Las variables definidas dentro de las funciones se llaman variables locales. Su valor sólo se puede utilizar dentro de la función en la que se declaran.
- Los parámetros formales son variables locales a la función
- Las variables declaradas fuera de una función se conocen como variables globales, nosotros no usaremos **VARIABLES GLOBALES** por lo que siempre usaremos pasaje de parámetros para intercambiar datos.

El **tiempo de vida** de una variable en Python es el tiempo que dura la variable en la memoria del programa.

Recordemos que:

- Las variables locales existen mientras la función se ejecuta.
- Las variables locales se destruyen cuando la función finaliza.

El **alcance** de una variable en Python es el conjunto de líneas de código(sentencias), en donde la variable puede ser accedida.

*"si la variable  $v$  esta definida en la línea  $n$ , el alcance de la variable  $v$  será cualquier línea  $m > n$ ".*

Esta situación se complica cuando tenemos funciones anidadas, es decir, funciones dentro de otras funciones.

*Consultas*

---

## Temas a desarrollar la próxima clase

- ☐ Repaso General
- ☐ Simulacro de 1er parcial